```
  # join the swarm
  docker-machine ssh do-worker-$i \
    "docker swarm join --token $SWARM_TOKEN $MANAGER_INTERNAL_IP:2377"
done
```

Deploying to Our DigitalOcean Swarm

We now have a three-node cluster in DigitalOcean. The next step is to deploy our app onto it. With Swarm, we use the same process whether we're deploying to a one-node cluster (as we did in the previous chapter), a three-node cluster (as we're about to do), or even a twenty-node cluster.

We start by configuring our Docker CLI to point to one of the manager nodes in the swarm (it's possible to have more than one manager). We can do this with `docker-machine` by setting the environment variables for the manager node:

```
$ eval $(docker-machine env do-manager-1)
```

Then, deploying our application is just a matter of running the `docker stack deploy` command. Let's do this now:

```
$ docker stack deploy -c docker-stack.yml myapp
Creating network myapp_default
Creating service myapp_db-migrator
Creating service myapp_web
Creating service myapp_redis
Creating service myapp_database
```

With our app deployed, we can see our services running on the Swarm cluster (although it may take a moment for them to all start):

```
$ docker service ls
ID   NAME               MODE  REP…  IMAGE                 PORTS
tj…  myapp_database     rep…  0/1   postgres:latest
zy…  myapp_db-migrator  rep…  0/1   robisenberg/myapp_w…
sx…  myapp_redis        rep…  0/1   redis:latest
vo…  myapp_web          rep…  0/1   robisenberg/myapp_w…  *:80->3000/tcp
```

We can also view the actual containers running for this stack (remember, it's possible to have more than one container running per service):

```
$ docker stack ps myapp
ID   NAME                IMAGE    …  DESIRED…  CURRENT…  …  …
gm…  myapp_database.1    postgr…  …  Running   Prepari…
g8…  myapp_redis.1       redis:…  …  Running   Startin…
xx…  myapp_web.1         robise…  …  Running   Prepari…
kh…  myapp_db-migrator…  robise…  …  Running   Prepari…
```